

Department of Electronic and Telecommunication Engineering

University of Moratuwa, Sri Lanka

System Bus Design

Multi-Master Shared Bus with Split Transactions
and UART Remote Bus Access

EN4021 – Advanced Digital Systems

MEMBERS:	INDEX No
Jayakody J.A.K.	220247J
Attanayake A.Y.	220051D
Rajinthan R.	220502M

Target board: Terasic DE2-115 (Intel Cyclone IV E, EP4CE115F29C7)

September 2026

Contents

1	Overview	3
1.1	Architecture	3
1.2	Key Features	3
2	Address Allocation	4
2.1	General Address Encoding	4
2.2	Memory Map	5
2.3	Addressing the Remote Board	5
3	Master Node	5
3.1	Purpose and Role	5
3.2	Block Diagram	6
3.3	I/O Description	6
3.4	State Diagram and Behaviour	6
4	UART Master Node	7
4.1	Purpose	7
4.2	Block Diagram	8
4.3	I/O Description	8
4.4	Wire Format	8
4.5	State Machine Logic	10
5	Slave Ports	10
5.1	Slave 0 — 4 KB Split RAM	10
5.1.1	Purpose	10
5.1.2	Block Diagram	10
5.1.3	I/O Description	10
5.1.4	Split Behaviour	10
5.2	Slaves 1 and 2 — Fast Single-Port RAM	12
5.2.1	Purpose	12
5.2.2	Block Diagram	12
5.2.3	I/O Description	13
6	Arbiter	13
6.1	Overview	13
6.2	I/O Description	13
6.3	State Machine	13
7	Address Decoder	14
7.1	Purpose	14

7.2 I/O Description	14
8 Bus Interconnect	15
8.1 Module	15
8.2 I/O Description	15
9 UART Modules	15
9.1 Two-Board Topology	16
10 System Integration	17
11 In-System Verification (JTAG ISSP)	17
11.1 Source Map (host → device, 50 bits)	19
11.2 Probe Map (device → host, 38 bits)	19
11.3 Test Procedure	19
11.4 Measured Latencies	19
12 Timing Analysis	19
13 Resource Utilisation	20
14 Simulation and Verification	21
14.1 Verification Strategy	21
14.2 Unit Testbenches	21
14.3 Integration Testbench: <code>tb_top_bus_system.v</code>	21
14.4 Two-Board Testbench: <code>tb_uart_remote.v</code>	22
14.5 Running the Simulations	22
15 Design Decisions	22
16 Known Limitations	23
17 Board Bring-Up	23
A Appendix: Selected RTL	24
A.1 <code>address_decoder.v</code>	24
A.2 <code>arbiter_2m_split.v</code>	25
A.3 <code>master_node.v</code>	26
A.4 <code>slave_0_split_4k.v</code>	28

1 Overview

The System Bus is a synchronous, multi-master shared interconnect for FPGA platforms. The implementation reported here targets the Intel Cyclone IV E family on the Terasic DE2-115 board and provides a two-master, three-slave memory-mapped bus with hardware split-transaction support and a UART link that lets one board execute bus transactions on a second, identical board.

The bus carries a 14-bit address, an 8-bit write data path and an 8-bit read data path, with an explicit request/grant arbitration scheme. Split transactions let the slow slave release the bus while it fetches, so the rest of the system continues to run instead of stalling behind it.

1.1 Architecture

The implemented design provides:

- **Two masters.** Master 0 (`master_node_uart`) is the high-priority master and additionally carries the UART link to the second board. Master 1 (`master_node`) is a plain local master at lower priority.
- **Three memory slaves.** Slave 0 is a 4 KB split-transaction RAM; Slaves 1 and 2 are single-cycle RAMs of 4 KB and 2 KB.
- **A central bus interconnect** (`bus_interconnect`) containing the arbiter, the address decoder, the granted-master multiplexer and the slave response return multiplexer.
- **Split transaction support** for the long-latency slave, so a 3-cycle read on Slave 0 does not block the other master.
- **UART remote bus access**, in which a command issued on Master 0 with `cmd_remote = 1` is executed on the other board's bus and its result returned.
- **A JTAG In-System Sources and Probes front-end** (`bus_issp_driver`) that drives both master command ports and reports results and latencies without using any board pins.

1.2 Key Features

- **Multi-master support.** Two masters with deterministic fixed-priority arbitration (Master 0 prioritised), with one override: a pending split whose data has arrived jumps the queue.
- **Split transactions.** Slave 0 cannot answer a read immediately. It pulses `split_req`, and the arbiter revokes the grant and hands the bus to Master 1 while Slave 0 fetches in the background. `split_done` brings the original master back.
- **UART remote bus access.** Master 0 is symmetric: it is the *client* for our own remote commands and the *server* for the other board's, and both run through one shared `master_node` core.
- **Fail-safe remote link.** A remote transaction that gets no answer within `RESP_TIMEOUT` completes with `cmd_error` set rather than hanging the bus.

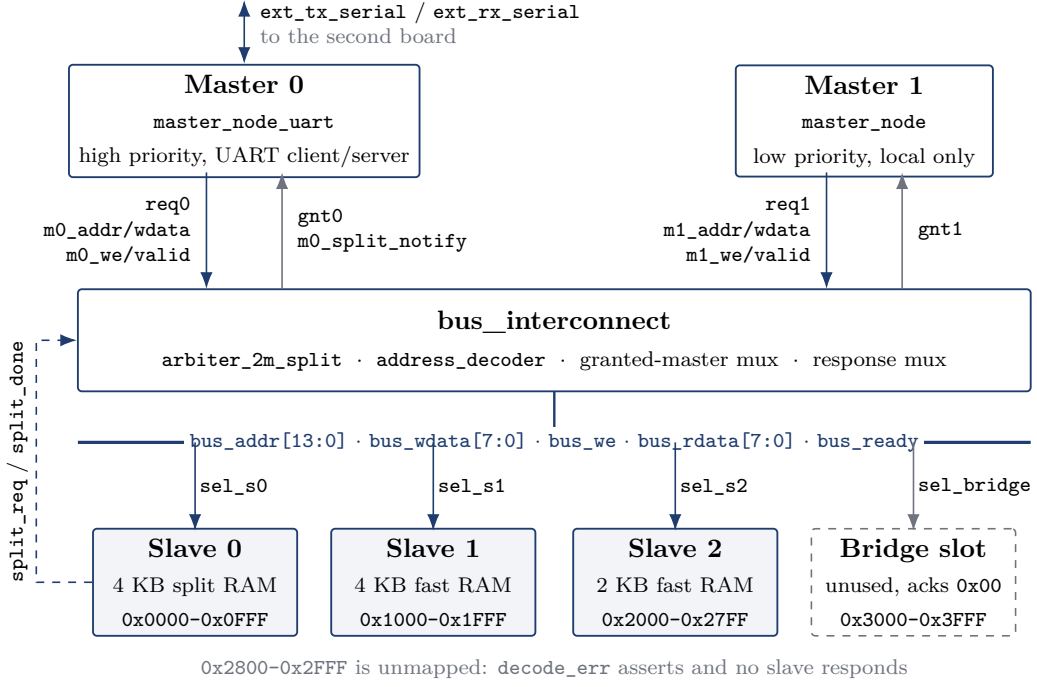


Figure 1: Full system architecture. One master owns the shared bus at a time; the interconnect multiplexes the granted master onto the bus and returns the selected slave’s response.

- **Embedded memory.** All three slave memories are full-depth `altsyncram` instances mapped into M9K blocks, so memory costs block RAM rather than logic.
- **Pin-free verification.** The whole bus is exercised over JTAG through In-System Sources and Probes; only twelve pins leave the device.

2 Address Allocation

2.1 General Address Encoding

A 14-bit address is used, giving a 16 KB address space. The slave is selected from the upper address bits by `address_decoder`; the remaining low-order bits index the word inside the selected memory. Slaves 0 and 1 consume `addr[11:0]` (4 K words each) and Slave 2 consumes `addr[10:0]` (2 K words).

Table 1: Decode fields

Match	Select	Target	Local address
<code>addr[13:12] == 2'b00</code>	<code>sel_s0</code>	Slave 0, 4 KB split RAM	<code>addr[11:0]</code>
<code>addr[13:12] == 2'b01</code>	<code>sel_s1</code>	Slave 1, 4 KB fast RAM	<code>addr[11:0]</code>
<code>addr[13:11] == 3'b100</code>	<code>sel_s2</code>	Slave 2, 2 KB fast RAM	<code>addr[10:0]</code>
<code>addr[13:12] == 2'b11</code>	<code>sel_bridge</code>	Bridge slot (unused)	—
anything else	<code>decode_err</code>	no target	—

Table 2: System memory map

Range	Size	Target	Latency	Notes
0x0000–0x0FFF	4 KB	Slave 0 (<code>slave_0_split_4k</code>)	write 1 clk, read split	Reads split; 3-cycle background fetch on a dual-port M9K.
0x1000–0x1FFF	4 KB	Slave 1 (<code>slave_fast_ram</code> , AW=12)	1 clk	Single-port M9K, <code>ready <= sel</code> .
0x2000–0x27FF	2 KB	Slave 2 (<code>slave_fast_ram</code> , AW=11)	1 clk	Same module, half the depth.
0x2800–0x2FFF	2 KB	<i>unmapped</i>	—	<code>decode_err</code> asserts; no slave answers (see Section 16).
0x3000–0x3FFF	4 KB	Bridge slot	1 clk	Decoder slot kept for the bridge; answered with 0x00 so it can never hang the bus.

2.2 Memory Map

2.3 Addressing the Remote Board

Remote access is *not* address-mapped. Rather than reserving a window and translating it, the command port of Master 0 carries an extra `cmd_remote` qualifier:

- `cmd_remote = 0` — the transaction runs on the local bus and the address means what Table 2 says.
- `cmd_remote = 1` — the identical 14-bit address is carried over the UART and executed against the *other board's* memory map, which is the same map on the same design.

The full 16 KB space is therefore reachable on either board, and a local address and its remote twin are written the same way. The historical `cross_fpga_bridge` slave that occupied 0x3000–0x3FFF has been removed; the decoder slot survives only so that an access there is acknowledged instead of hanging.

3 Master Node

3.1 Purpose and Role

`master_node` is the interface between a local command source (the ISSP driver, or the UART server logic) and the shared bus. It latches a command, raises `bus_req`, waits for the grant, drives address, write data and write enable onto its master port, and returns read data with a one-cycle `cmd_done` pulse. It also implements the master side of the split protocol.

3.2 Block Diagram

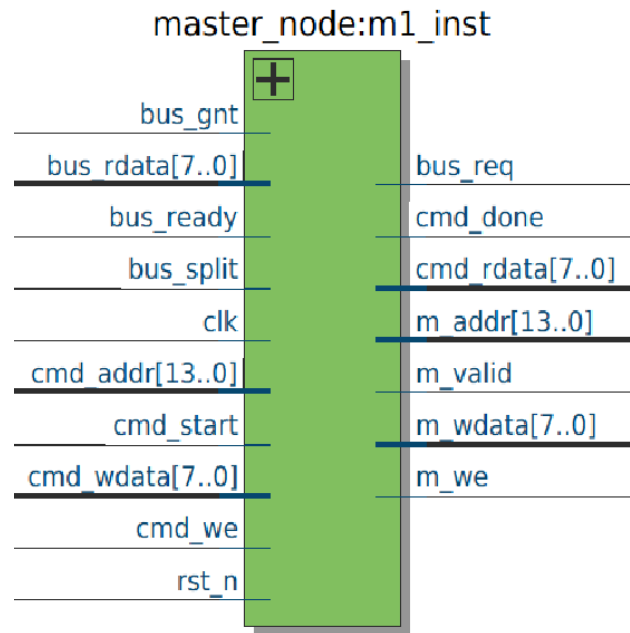


Figure 2: master_node as elaborated by the Quartus RTL viewer (instance m1_inst).

3.3 I/O Description

3.4 State Diagram and Behaviour

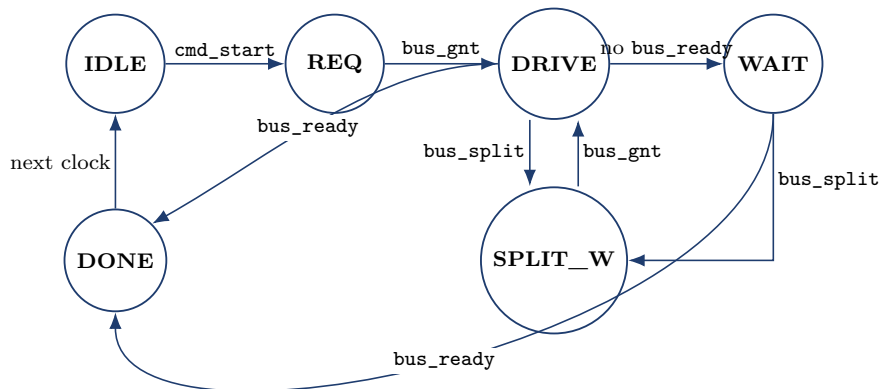


Figure 3: master_node state machine. After a split the master returns to DRIVE on the re-grant and completes there.

- **IDLE.** Waits for cmd_start. On the strobe it latches address, data and direction into internal registers, asserts bus_req and moves to REQ.
- **REQ.** Waits for bus_gnt. On the grant it drives m_addr, m_wdata, m_we and raises m_valid.

Table 3: master_node I/O

Signal	Dir	Description
clk, rst_n	Input	50 MHz clock; active-low asynchronous reset.
cmd_start	Input	One-cycle strobe that launches a transaction.
cmd_we	Input	1 = write, 0 = read.
cmd_addr[13:0]	Input	Target bus address.
cmd_wdata[7:0]	Input	Write data.
cmd_rdata[7:0]	Output	Read data, captured on reads only and held until the next read.
cmd_done	Output	One-cycle completion pulse.
bus_req	Output	Request to the arbiter; held through a split.
bus_gnt	Input	Grant from the arbiter.
bus_split	Input	Split notification from the arbiter.
m_addr[13:0]	Output	Address driven to the master multiplexer.
m_wdata[7:0]	Output	Write data driven to the master multiplexer.
m_we	Output	Write enable driven to the master multiplexer.
m_valid	Output	Qualifies the master port; gates the decoder.
bus_rdata[7:0]	Input	Read data return bus.
bus_ready	Input	Target acknowledge.

- **DRIVE.** If `bus_split` is asserted it drops `m_valid`, *keeps* `bus_req` high and parks in `SPLIT_W`. If `bus_ready` is asserted the transaction completes: on a read the return data is captured, `cmd_done` pulses, and the FSM goes to `DONE`. Otherwise it waits.
- **WAIT.** Same two exits as `DRIVE`, for slaves that need more than one cycle to answer.
- **SPLIT_W.** Holds `bus_req` while the slave fetches in the background. When the arbiter re-grants, the FSM re-issues the read from its latched address (forced to a read, write data zeroed) and returns to `DRIVE` to collect the prepared data.
- **DONE.** Clears the control flags and returns to `IDLE`.

Read data is captured only when `reg_we` is 0. The slave memories echo `din` onto `dout` during a write, so capturing unconditionally would overwrite the last read value with the byte just written — and, since `cmd_rdata` drives the LEDs, would visibly corrupt the display after every write.

4 UART Master Node

4.1 Purpose

`master_node_uart` is Master 0. It wraps an ordinary `master_node` core and adds a UART client and a UART server around it, so that one block both issues remote transactions and executes the other board's. There is no separate bridge master and bridge slave: the same core performs every local bus access, whoever asked for it.

- **Client path:** `cmd_remote` → send REQUEST → wait for RESPONSE → `cmd_rdata`, `cmd_done`.
- **Server path:** REQUEST arrives → run it on the local bus through the core → send RESPONSE.

4.2 Block Diagram

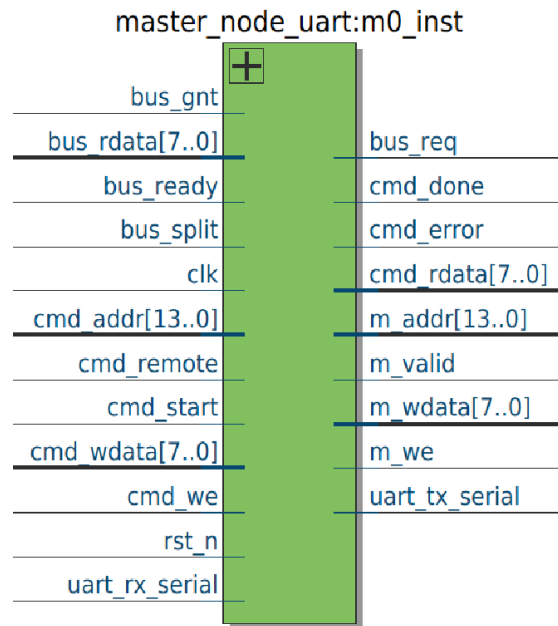


Figure 4: `master_node_uart` (instance `m0_inst`). Beyond the ordinary master ports it carries `cmd_remote`, `cmd_error` and the two serial pins.

4.3 I/O Description

Table 4: `master_node_uart` I/O (additions to `master_node`)

Signal	Dir	Description
<code>cmd_remote</code>	Input	1 = execute this transaction on the other board.
<code>cmd_error</code>	Output	Set with <code>cmd_done</code> when a remote transaction timed out.
<code>uart_rx_serial</code>	Input	Serial input from the other board.
<code>uart_tx_serial</code>	Output	Serial output to the other board.
<code>remote_busy</code>	Output	A remote transaction is outstanding.
<code>srv_busy</code>	Output	Currently serving the other board's request.
<i>Parameters</i>		
<code>CLKS_PER_BIT</code>	—	Baud divisor, default 434 (50 MHz / 115200).
<code>RESP_TIMEOUT</code>	—	Response timeout in clocks, default 500 000 (≈ 10 ms).

All other ports are identical to `master_node` and pass straight through to the core.

4.4 Wire Format

Frames are tagged so a request and a response can never be confused, which matters because both directions share one physical link.

Bytes go out least-significant first. A write brings back a response carrying 0x00 as an acknowledgement, so `cmd_done` on a remote write means the far side really performed it.

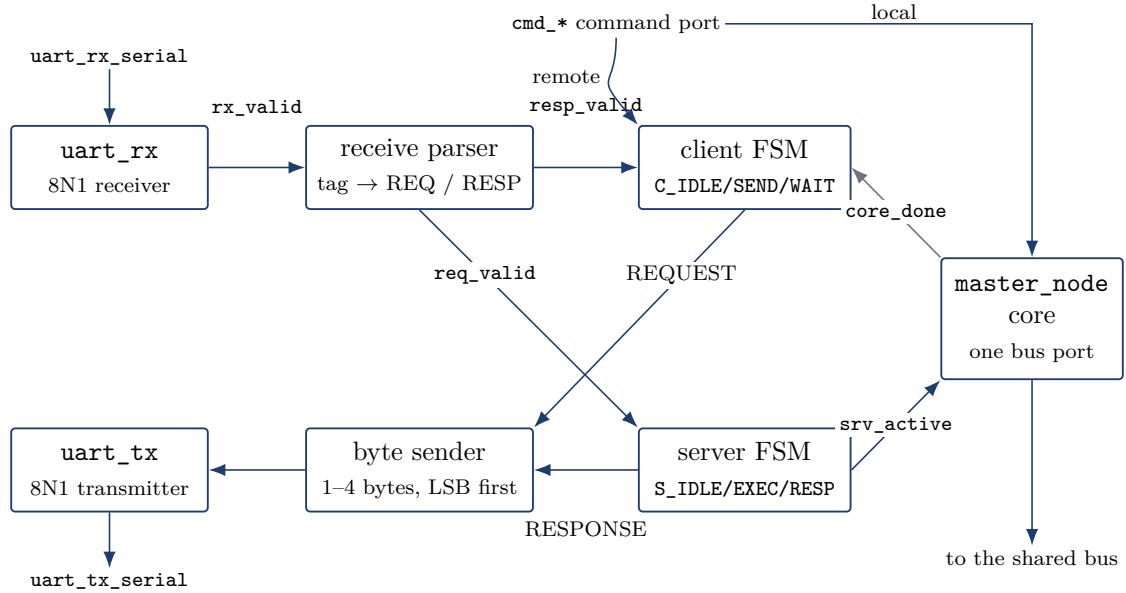


Figure 5: Internal structure of `master_node_uart`. `srv_active` multiplexes the command inputs of the shared core between the local command port and an incoming remote request.

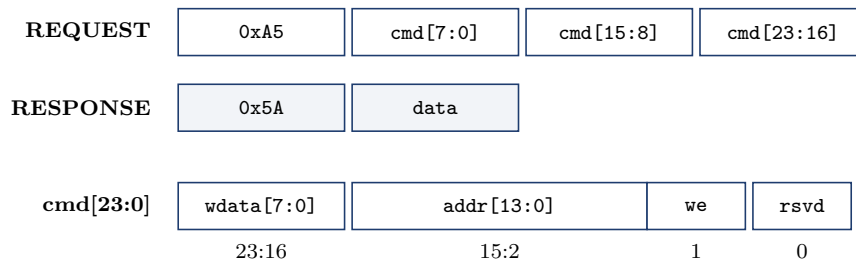


Figure 6: UART frame formats (8N1) and the 24-bit command layout, which uses the same field order as one ISSP source slice.

4.5 State Machine Logic

Client (**cs**).

- **C_IDLE**: on `cmd_start` with `cmd_remote`, packs `{cmd_wdata, cmd_addr, cmd_we, 1'b0}` into `out_cmd`.
- **C_SEND**: once the shared transmitter is free, queues the four-byte request and clears the timeout counter.
- **C_WAIT**: completes on `resp_valid` with the returned byte, or after `RESP_TIMEOUT` clocks with `cmd_rdata = 0xFF` and `cmd_error` set.

Server (**ss**).

- **S_IDLE**: a latched incoming request is taken up when the core is not already running a local command; `srv_active` switches the core's command inputs to the remote fields.
- **S_EXEC**: waits for `core_done`; a read captures `core_rdata`, a write yields `0x00`.
- **S_RESP**: queues the two-byte response and releases the core.

Transmitter priority. Responses take priority over requests in the shared byte sender (**C_SEND** is held while `ss == S_RESP`). Without that rule, two boards that issue remote commands on the same clock would both sit in **C_WAIT** and neither would ever answer the other.

5 Slave Ports

5.1 Slave 0 — 4 KB Split RAM

5.1.1 Purpose

`slave_0_split_4k` models a long-latency target. Writes complete in one cycle, but a read takes `SPLIT_LATENCY = 3` background cycles, during which the slave gives the bus up rather than stalling it.

The memory is one `altsyncram` in simple dual-port mode: port A is the write path driven by the live bus address, and port B is the read path driven by the internally latched `saved_raddr`. Mapping the two logically independent accesses onto the block's two real ports keeps the whole 4 K×8 memory inside M9K blocks instead of spilling into logic.

5.1.2 Block Diagram

5.1.3 I/O Description

5.1.4 Split Behaviour

1. A read arrives. The slave latches `addr` into `saved_raddr`, sets `processing_split`, pulses `split_req` and starts the latency counter.
2. The counter runs in the background. Port B of the M9K has had a stable address for the whole window, well beyond its one-cycle read latency, so at the terminal count `ram_q_b` is captured into `split_data_reg`, `split_done` pulses and `data_ready_for_fetch` is set.

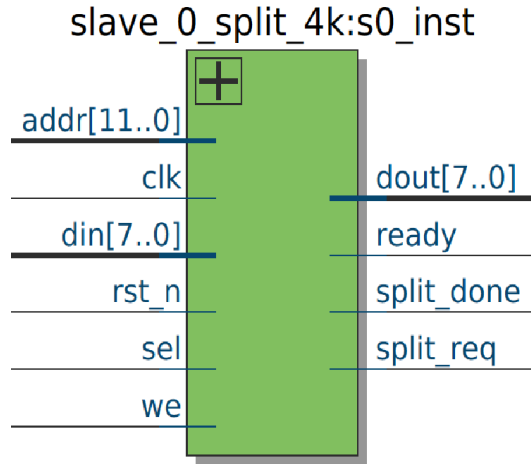


Figure 7: slave_0_split_4k (instance s0_inst). The split_req/split_done pair is what distinguishes it from the fast slaves.

Table 5: slave_0_split_4k I/O

Signal	Dir	Description
clk, rst_n	Input	Clock and active-low reset.
sel	Input	Select from the decoder (sel_s0).
we	Input	1 = write, 0 = read.
addr[11:0]	Input	Word address inside the 4 KB window.
din[7:0]	Input	Write data.
dout[7:0]	Output	Read data, or the write echo.
ready	Output	Transaction acknowledge.
split_req	Output	One-cycle strobe asking the arbiter to split.
split_done	Output	One-cycle strobe when the background fetch has landed.

3. The master is re-granted and re-issues the same address. It matches `saved_raddr`, so the slave returns `split_data_reg` with `ready` and clears `data_ready_for_fetch`.

The `just_completed` flag blocks the split-start branch for exactly one cycle after a split is served. Without it, the master's `m_valid` (and therefore `sel`) is still high on the following cycle, the start condition matches again, and the slave opens a phantom second split for the same address — which leaves a later read of any *other* slave-0 address matching neither branch, asserting neither `ready` nor `split_req`, and deadlocking the bus.

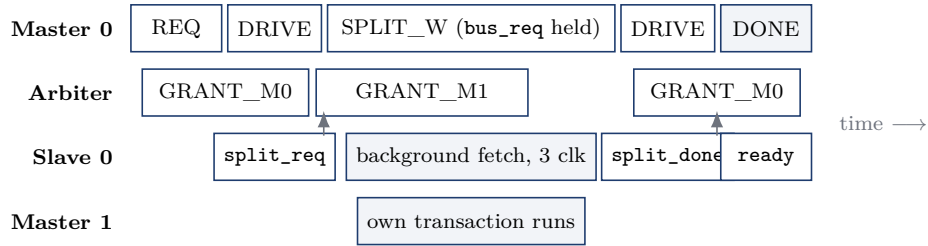


Figure 8: Split read on Slave 0. Master 1 gets real work done inside the window Master 0 spends waiting; the lane widths are indicative, not exact cycle counts.

5.2 Slaves 1 and 2 — Fast Single-Port RAM

5.2.1 Purpose

`slave_fast_ram` is the ordinary target: a single-port `altsyncram` in `NEW_DATA_NO_NBE_READ` mode, which gives exactly “`dout = din` on a write, `dout = stored word` on a read” one cycle later, with no register stage stacked on top of the block’s own synchronous read latency. `ready` is a registered copy of `sel`, so every access completes in one cycle. Depth is set by the `ADDR_WIDTH` parameter — 12 for the 4 KB Slave 1, 11 for the 2 KB Slave 2.

5.2.2 Block Diagram

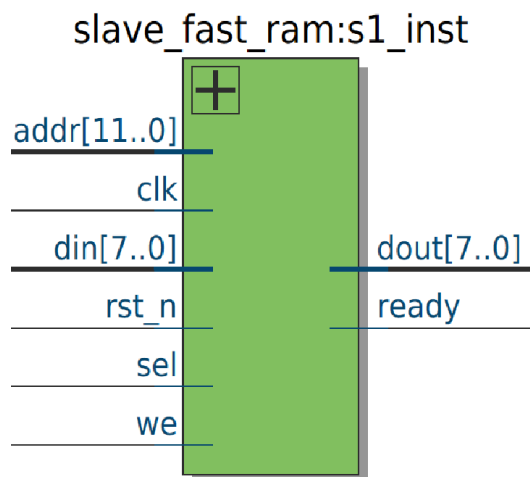


Figure 9: `slave_fast_ram` (instance `s1_inst`). Slave 2 is the same module with `ADDR_WIDTH = 11`.

Table 6: slave_fast_ram I/O

Signal	Dir	Description
clk, rst_n	Input	Clock and active-low reset.
sel	Input	Select from the decoder (<code>sel_s1</code> or <code>sel_s2</code>).
we	Input	Write enable.
addr[AW-1:0]	Input	Local word address.
din[7:0]	Input	Write data.
dout[7:0]	Output	Read data, straight from the M9K output.
ready	Output	One-cycle acknowledge, <code>ready <= sel</code> .

5.2.3 I/O Description

6 Arbiter

6.1 Overview

`arbiter_2m_split` serialises access to the shared bus. Priority is fixed: Master 0 over Master 1. The one override is the split path — a Master 0 split whose data has arrived jumps ahead of a standing Master 1 request, because otherwise the completed fetch could be starved indefinitely.

6.2 I/O Description

Table 7: arbiter_2m_split I/O

Signal	Dir	Description
clk, rst_n	Input	Clock and active-low reset.
req0, req1	Input	Requests from Master 0 (high priority) and Master 1.
gnt0, gnt1	Output	Grants to Master 0 and Master 1.
s0_split_req	Input	Split request from Slave 0.
s0_split_done	Input	Background completion strobe from Slave 0.
m0_split_notify	Output	Tells Master 0 its transaction has been split.
m1_split_notify	Output	Declared for symmetry; never asserted (Section 16).

Two internal status bits carry the split across states: `m0_split_pending` is set when a split starts under `GRANT_M0`, and `split_data_ready` latches the `s0_split_done` pulse. Both clear when Master 0 re-acquires the bus and completes.

6.3 State Machine

- **IDLE.** Evaluated in strict order: a pending split whose data is ready wins; then a plain Master 0 request (suppressed while a split of its own is pending); then Master 1.
- **GRANT_M0.** Drives `gnt0`. If Slave 0 raises `s0_split_req`, the arbiter asserts `m0_split_notify` and immediately hands the bus to Master 1 if it wants it, otherwise returns to IDLE. It also leaves when Master 0 drops its request.
- **GRANT_M1.** Drives `gnt1` and holds it until Master 1 finishes; only then is a pending Master 0 split or request serviced. Master 1 is never pre-empted mid-transaction.

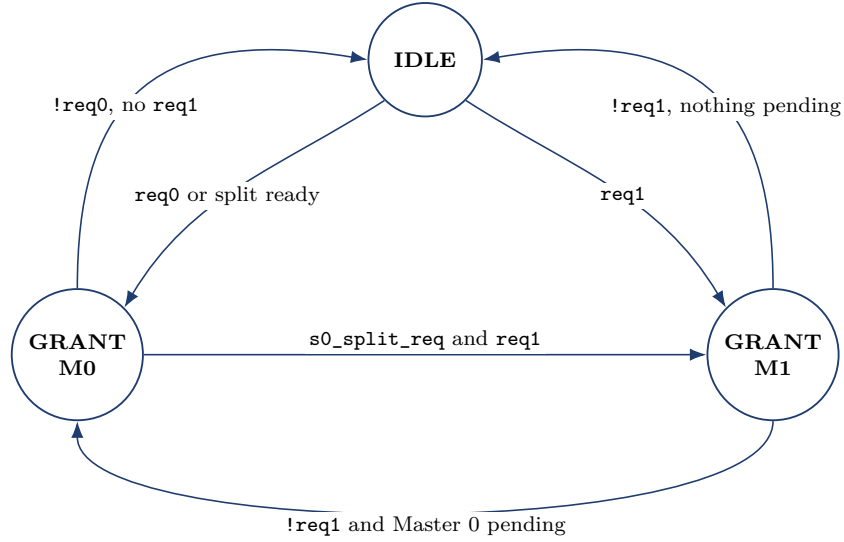


Figure 10: `arbiter_2m_split` state machine. From IDLE the exits are evaluated in priority order each cycle: a split whose data is ready, then `req0`, then `req1`.

Because Master 0 is granted first, it often *finishes* first even on the slow split slave, with Master 1 absorbing the wait. That latency ordering is an artefact of arbitration, not a property the protocol guarantees.

7 Address Decoder

7.1 Purpose

`address_decoder` is purely combinational. Whenever `bus_valid` is high it decodes the upper address bits into one-hot slave selects; when `bus_valid` is low every select is low, so a bus with no granted master touches no slave.

7.2 I/O Description

Table 8: `address_decoder` I/O

Signal	Dir	Description
<code>addr[13:0]</code>	Input	Address of the granted master.
<code>bus_valid</code>	Input	<code>m_valid</code> of the granted master; qualifies the decode.
<code>sel_s0</code>	Output	Slave 0 select, 0x0000–0x0FFF.
<code>sel_s1</code>	Output	Slave 1 select, 0x1000–0x1FFF.
<code>sel_s2</code>	Output	Slave 2 select, 0x2000–0x27FF.
<code>sel_bridge</code>	Output	Bridge slot select, 0x3000–0x3FFF.
<code>decode_err</code>	Output	Unmapped access, 0x2800–0x2FFF.

The decode is the priority chain of Table 1: two-bit matches for Slaves 0 and 1 and the bridge slot, a three-bit match for the half-size Slave 2, and `decode_err` as the fall-through.

8 Bus Interconnect

8.1 Module

`bus_interconnect` is the fabric. It instantiates the arbiter and the decoder and owns the three multiplexers that make a shared bus out of them:

1. **Master multiplexer.** `bus_addr`, `bus_wdata`, `bus_we` and `bus_valid` follow `gnt0`, then `gnt1`, and are driven to zero when neither is granted.
2. **Decode.** The multiplexed address and valid drive `address_decoder`, producing the one-hot selects.
3. **Response multiplexer.** `bus_rdata` and `bus_ready` are taken from whichever slave is selected, and are `0x00` and `0` when none is.

Splitting this out of the top level keeps `top_bus_system` to nothing but instantiation and wiring, and means the arbitration and decode policy live in one file.

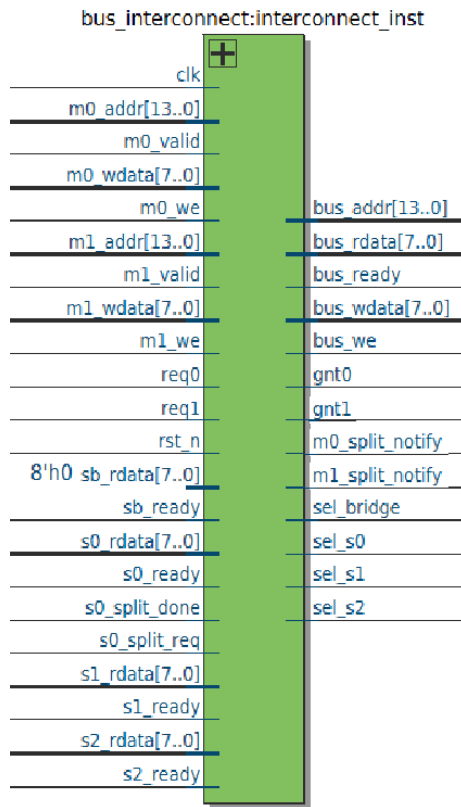


Figure 11: `bus_interconnect`. Both master port groups and every slave return path enter on the left; the shared bus, the grants, the split notifications and the one-hot selects leave on the right.

8.2 I/O Description

9 UART Modules

The physical link uses a plain 8N1 UART with a parameterised baud divisor, `CLKS_PER_BIT =  $f_{\text{clk}}/\text{baud}$` , which is 434 for 115 200 baud at 50 MHz. Both halves are four-state machines (IDLE, START, DATA, STOP).

Table 9: bus_interconnect I/O

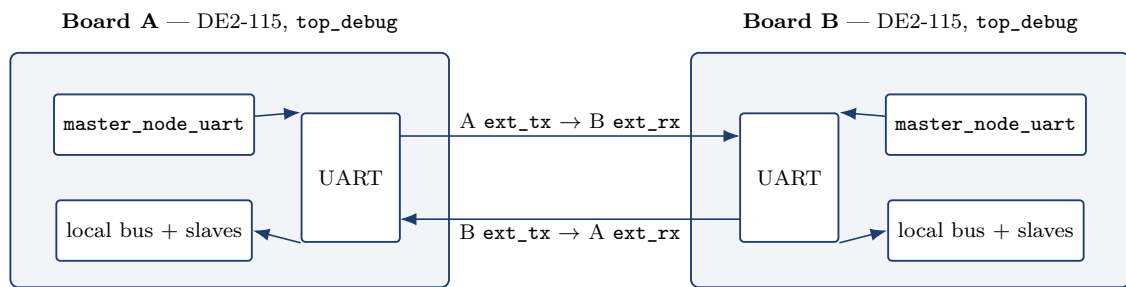
Signal	Dir	Description
req0, req1	Input	Requests from the two masters.
gnt0, gnt1	Output	Grants back to the masters.
m0_split_notify, m1_split_notify	Output	Split notifications from the arbiter.
m0_addr, m0_wdata, m0_we, m0_valid	Input	Master 0 port.
m1_addr, m1_wdata, m1_we, m1_valid	Input	Master 1 port.
bus_addr, bus_wdata, bus_we	Output	Shared bus driven to all slaves.
sel_s0, sel_s1, sel_s2, sel_bridge	Output	One-hot slave selects.
s0_split_req, s0_split_done	Input	Split handshake from Slave 0 to the arbiter.
s0/s1/s2/sb_rdata, _ready	Input	Per-slave return data and acknowledge.
bus_rdata, bus_ready	Output	Multiplexed response to the masters.

uart_rx double-flops rx_serial before use, since it arrives from a pin and is asynchronous to clk; it samples each bit at mid-bit and pulses rx_valid for one cycle when a byte has been framed.

Table 10: uart_tx and uart_rx I/O

Module	Signal	Dir	Description
uart_tx	tx_start	Input	One-cycle strobe; tx_data must be valid.
	tx_data	Input	Byte to transmit.
	tx_serial	Output	Serial line, idles high.
	tx_busy	Output	High until the stop bit has been driven.
uart_rx	rx_serial	Input	Serial line from the other board.
	rx_data	Output	Received byte.
	rx_valid	Output	One-cycle strobe when a byte is framed.

9.1 Two-Board Topology



AC15 to AB22 on each board, plus a common ground

Figure 12: Two identical boards with a crossed serial link. Either board can be client or server, and both roles run concurrently.

10 System Integration

`top_bus_system` instantiates the two masters, the interconnect and the three slaves, and wires the bridge slot's acknowledge. `top_debug` is the synthesis top: it adds `bus_issp_driver` and exposes only `clk`, `rst_n`, the two serial pins and `led[7:0]`.

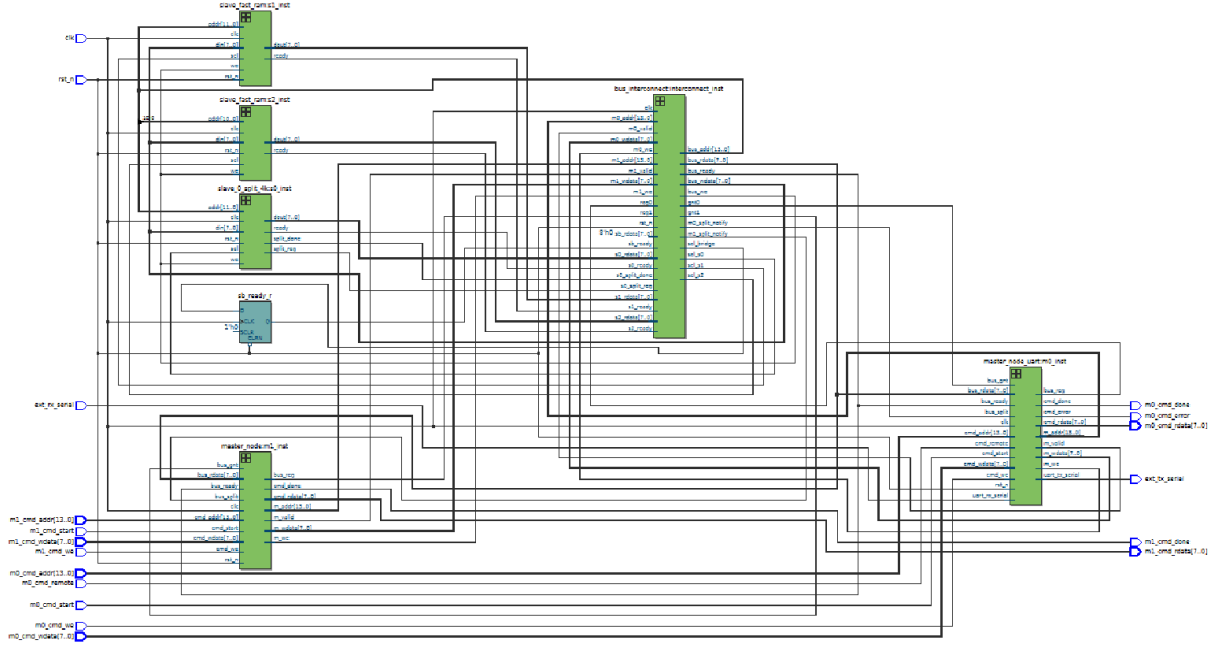


Figure 13: Quartus RTL netlist of `top_bus_system` as instantiated on the device: both masters, the interconnect, the three slave memories and the registered acknowledge for the unused bridge slot.

The bridge slot deserves a note. The UART now lives inside Master 0, so nothing is attached to `sel_bridge`. Leaving it dangling would make 0x3000–0x3FFF behave exactly like the unmapped gap, so the slot is given a registered acknowledge returning 0x00:

```
1 reg sb_ready_r;
2 always @(posedge clk or negedge rst_n) begin
3     if (!rst_n) sb_ready_r <= 1'b0;
4     else sb_ready_r <= sel_bridge;
5 end
6 assign sb_rdata = 8'h00;
7 assign sb_ready = sb_ready_r;
```

Listing 1: Bridge slot acknowledge in `top_bus_system`

`led[7:0]` shows `m0_cmd_rdata`, which the master updates on reads only, so the display holds the last byte read by Master 0.

11 In-System Verification (JTAG ISSP)

Routing 14 address lines and 8 data lines to switches was not practical, so both master command ports are driven over JTAG through an Altera In-System Sources and Probes instance (`altsource_probe`, instance ID "BUS0"), with a 50-bit source and a 38-bit probe.

Table 11: ISSP source bit map

Bits	Field	Description
src[0]	m0_go	Level; a 0→1 edge launches one Master 0 command.
src[1]	m0_cmd_we	1 = write, 0 = read.
src[15:2]	m0_cmd_addr	14-bit bus address.
src[23:16]	m0_cmd_wdata	8-bit write data.
src[24]	m1_go	Master 1 launch level.
src[25]	m1_cmd_we	Master 1 direction.
src[39:26]	m1_cmd_addr	Master 1 address.
src[47:40]	m1_cmd_wdata	Master 1 write data.
src[48]	soft_rst	Clears the driver's sticky status flags.
src[49]	m0_remote	1 = execute this command on the other board.

Table 12: ISSP probe bit map

Bits	Field	Description
prb[7:0]	m0_r1	Last read data captured for Master 0.
prb[8]	m0_done_s	Sticky “command finished”; clears when m0_go drops.
prb[9]	m0_busy	Command in flight.
prb[17:10]	m1_r1	Last read data for Master 1.
prb[18]	m1_done_s	Master 1 sticky done.
prb[19]	m1_busy	Master 1 busy.
prb[27:20]	m0_lat	Clocks from launch to done, saturating at 0xFF.
prb[35:28]	m1_lat	Same for Master 1.
prb[36]	collision	Sticky: both masters were in flight at once.
prb[37]	m0_error	Sticky: the last remote command timed out.

11.1 Source Map (host → device, 50 bits)

11.2 Probe Map (device → host, 38 bits)

A latency reading of 0xFF with busy still high means the transaction never completed — for instance an address in the 0x2800–0x2FFF decode gap, which asserts no select and therefore never returns `bus_ready`.

11.3 Test Procedure

`issp_bus_test.tcl` replays the `tb_top_bus_system.v` cases against the real board and exits 0 on pass, 1 on failure. `issp_console.tcl` is the interactive counterpart (`w / r / both / sweep` against any slave and offset); `both` fires both masters from a single source write so they reach the arbiter on the same clock. Connection handling and the bit map are shared through `issp_bus_lib.tcl`.

```
cd System_Bus_Final
quartus_stp -t issp_bus_test.tcl # the standard test run
quartus_stp -t issp_bus_test.tcl -gap # + decode-gap deadlock demo
```

Listing 2: Running the hardware tests

`quartus_stp` is the only interpreter that works. The Tcl packages `::quartus::jtag` and `::quartus::insystem_source_probe` are rejected outright by `quartus_sh`, and are absent from the GUI Tcl console. To launch from the GUI, `run_issp_test.tcl` shells out to `quartus_stp` and echoes the output back.

Two ordering constraints inside the script proved easy to trip over:

- `get_insystem_source_probe_instance_info` opens a transient JTAG session of its own, so it must be called *before* `start_insystem_source_probe`, never after.
- Bit 0 of each 24-bit source slice is the `go` level and is deliberately excluded from the command payload (`src[23:1]`, not `src[23:0]`). Widening it back over bit 0 aliases `we` onto `go` and makes reads impossible.

The In-System Sources and Probes editor tab must be closed before a scripted run, since an open editor holds the JTAG session.

11.4 Measured Latencies

Table 13: Transaction latency measured on hardware via `m0_lat`

Transaction	Clocks	At 50 MHz
Fast slave read (Slave 1 or 2), uncontended	6	120 ns
Split slave read (Slave 0), uncontended	12	240 ns

12 Timing Analysis

The project as committed contains no `.sdc` file, so the only clock the Timing Analyzer sees is the JTAG `altera_reserved_tck` and the system clock is left unconstrained. For this report the fitted design was re-analysed with the constraint below, applied to a scratch copy of the project.

```

create_clock -name clk -period 20.000 [get_ports {clk}]
derive_pll_clocks
derive_clock_uncertainty
set_false_path -from [get_ports {rst_n}]
set_false_path -from [get_ports {ext_rx_serial}]
set_false_path -to [get_ports {ext_tx_serial}]
set_false_path -to [get_ports {led[*]}]

```

Listing 3: Timing constraints used (`System_Bus_Final.sdc`)

Table 14: Timing results for `clk` constrained at 50 MHz (Quartus Prime Lite 24.1std, EP4CE115F29C7)

Check	Slow 1200 mV 85 °C	Slow 1200 mV 0 °C	Fast 1200 mV 0 °C
Setup slack	12.273 ns	12.923 ns	16.186 ns
Hold slack	0.402 ns	0.353 ns	0.181 ns
Minimum pulse width	9.554 ns	9.560 ns	9.201 ns

Total negative slack is 0.000 ns on every check at every corner. The worst-case corner reports $F_{\max} = \mathbf{129.42\text{ MHz}}$ for `clk`. Checked against the 20 ns period directly,

$$T_{\min} = 20\text{ ns} - 12.273\text{ ns} = 7.727\text{ ns}, \quad F_{\max} \approx \frac{1}{7.727\text{ ns}} \approx 129\text{ MHz},$$

which agrees with the tool. The design meets 50 MHz with roughly $2.6\times$ margin, and hold is met at every corner. Adding this `.sdc` to the project is the single most worthwhile follow-up: without it, timing is never actually signed off by the flow.

13 Resource Utilisation

Table 15: Fitter resource usage, `top_debug` on EP4CE115F29C7

Resource	Used / available	Utilisation
Total logic elements	1,097 / 114,480	< 1%
Combinational functions	734 / 114,480	< 1%
Dedicated logic registers	796 / 114,480	< 1%
Total registers	796	—
Total pins	12 / 529	2%
Total memory bits	81,920 / 3,981,312	2%
Embedded multipliers (9-bit)	0 / 532	0%
Total PLLs	0 / 4	0%

Table 16: M9K block usage by slave

Instance	Mode	Depth $\times$ width	Bits	M9Ks
<code>s0_inst (slave_0_split_4k)</code>	Simple dual port	4096×8	32,768	4
<code>s1_inst (slave_fast_ram)</code>	Single port	4096×8	32,768	4
<code>s2_inst (slave_fast_ram)</code>	Single port	2048×8	16,384	2
Total			81,920	10

All 10 KB of bus memory sits in M9K blocks, which is why logic use stays under 1% on a device this size. An earlier revision reset the memories with a loop over all 4096 words; that forces the synthesiser to build them from LUTs and registers instead, and was removed.

14 Simulation and Verification

14.1 Verification Strategy

Verification is bottom-up. Every module has its own self-checking testbench that prints a banner and an error count; two system-level testbenches then exercise the assembled bus and the two-board link; hardware runs through ISSP close the loop on the real device.

The slave memories instantiate `altsyncram`, which Quartus's own `altera_mf.v` cannot supply to `iverilog` (it does not parse), so simulation uses a behavioural stub for that megafunction.

14.2 Unit Testbenches

Table 17: Unit-level testbenches and results

Testbench	Result	Coverage
<code>tb_address_decoder.v</code>	PASS	Every range boundary, both edges of each window, the decode gap, and <code>bus_valid = 0</code> .
<code>tb_arbiter_2m_split.v</code>	PASS	Priority order, simultaneous requests, split release and re-grant.
<code>tb_master_node.v</code>	PASS	Write, read, split suspend and resume, back-to-back commands.
<code>tb_slave_0_split_4k.v</code>	PASS	Single-cycle write, split request, background completion, data fetch.
<code>tb_slave_fast_ram.v</code>	PASS	Write and read, boundary addresses, deselect behaviour.

14.3 Integration Testbench: `tb_top_bus_system.v`

The system testbench drives both master command ports of `top_bus_system` and checks returned data against a reference model, with randomised addresses and data:

- **Test 1 — Master 0 fast RAM.** 100 randomised write/read pairs on Slave 1.
- **Test 2 — Master 1 fast RAM.** The same from the low-priority master, confirming it is granted and served correctly.
- **Test 3 — Unmapped bridge slot.** 100 randomised accesses to `0x3000-0x3FFF`; every one is acknowledged with `0x00` and none hangs.
- **Test 4 — Split RAM interleaved.** 100 randomised operations on Slave 0 interleaved with the other master, exercising split, release, re-grant and data return under contention.

```
=====
STARTING TOP BUS SYSTEM INTEGRATION VERIFICATION
=====
[TEST 1] M0 Fast RAM Write & Read (100 Randomized Trials)...
[TEST 2] M1 Fast RAM Write & Read (100 Randomized Trials)...
[TEST 3] Unmapped Slave-3 Slot (100 Randomized Trials)...
-> SUCCESS: unmapped slot always acknowledged with 0x00 (no hang)
[TEST 4] Split RAM Interleaved Operations (100 Randomized Trials)...
```

```
=====
>> TOP SYSTEM TEST PASSED: All Interconnect Operations Correct! <<
=====
```

Listing 4: `tb_top_bus_system.v` console output

14.4 Two-Board Testbench: `tb_uart_remote.v`

This testbench instantiates two complete `top_bus_system` instances with their serial links crossed, shrinking `CLKS_PER_BIT` and `RESP_TIMEOUT` so a full remote transaction fits in a reasonable simulation. It covers remote write, remote read, both directions, a remote read of the split slave, and link failure.

```
[3] A performs a REMOTE WRITE into B's 0x1000...
-> SUCCESS: B local 0x1000 = 0x5a
[4] A's own 0x1000 must be untouched...
-> SUCCESS: A local 0x1000 = 0x11
[5] A performs a REMOTE READ of B's 0x1000...
-> SUCCESS: A remote read of B 0x1000 = 0x5a
[6] Reverse direction: B remote-reads A's slave 2 (0x2000)...
-> SUCCESS: B remote read of A 0x2000 = 0xc7
[7] A remote-reads B's SPLIT slave (0x0010)...
-> SUCCESS: A remote read of B split slave = 0x7e
[8] Unplug the link - a remote command must time out, not hang...
-> SUCCESS: remote read reported cmd_error instead of hanging
[9] Local access still works with the link down...
-> SUCCESS: A local 0x1000 = 0x11
=====
>> REMOTE ACCESS TEST PASSED: both boards read/write each other <<
=====
```

Listing 5: `tb_uart_remote.v` console output (abridged)

Test 8 is the one worth calling out: it proves the timeout path, so a pulled cable produces a failed transaction with `cmd_error` rather than a bus that never releases.

14.5 Running the Simulations

```
# full integration
iverilog -g2005 -o top.vvp tb_top_bus_system.v top_bus_system.v \
  bus_interconnect.v address_decoder.v arbiter_2m_split.v master_node.v \
  slave_0_split_4k.v slave_fast_ram.v master_node_uart.v uart_tx.v uart_rx.v
vvp top.vvp | grep -E "ERROR|FAILED|PASSED"

# two-board remote access
iverilog -g2005 -o rem.vvp tb_uart_remote.v top_bus_system.v \
  bus_interconnect.v address_decoder.v arbiter_2m_split.v master_node.v \
  master_node_uart.v slave_0_split_4k.v slave_fast_ram.v uart_tx.v uart_rx.v
vvp rem.vvp
```

Listing 6: Simulation commands

The testbenches exit 0 regardless of outcome, so the output must be checked rather than the exit status.

15 Design Decisions

Shared multiplexed bus rather than a crossbar. With two masters and three slaves, a crossbar would cost roughly 2×3 data paths and their arbitration for a concurrency this

workload does not need. A single granted master keeps the fabric to two multiplexers and one arbiter, and makes the split protocol expressible at all — there is a single bus to release.

Dual-port memory for the split slave. Slave 0 must accept a write on the live bus address while a background read of a different, latched address is in flight. Simple dual-port mode gives exactly those two independent accesses in one M9K, instead of extra logic to arbitrate a single port.

Memory sizing. Slave 2 is 2 KB, so its decode needs three address bits where the others need two, and it leaves the 0x2800–0x2FFF hole. The asymmetry is deliberate: it exercises non-uniform decoding and provides a genuinely unmapped region to test against.

Acknowledging the unmapped bridge slot. Since the bridge slave was removed, `sel_bridge` has nothing behind it. It is answered with 0x00 rather than left dangling, because an unanswered select hangs the bus exactly like the decode gap.

Read-only data capture in the master. Slaves echo `din` onto `dout` during a write. Capturing `bus_rdata` unconditionally would overwrite the last read value after every write, so capture is gated on `!reg_we`.

Response priority on the shared transmitter. Two boards issuing remote commands on the same clock would deadlock if requests could block responses; giving responses the transmitter first makes the link symmetric and self-clearing.

16 Known Limitations

The following are known, and are documented rather than hidden.

- **A decode-gap access deadlocks the bus.** `decode_err` is driven by `address_decoder` but left unconnected inside `bus_interconnect`. An access to 0x2800–0x2FFF asserts no select, so `bus_ready` never rises and the master waits forever holding `bus_req`. There is no timeout; only `rst_n` (KEY[0]) recovers it. `soft_rst` clears the ISSP driver’s status flags, not the fabric.
- **Slave 0 is effectively Master-0-only.** `m1_split_notify` is declared but never assigned 1; the arbiter reacts to `s0_split_req` solely in `GRANT_M0`. Master 1 reading 0x0000–0x0FFF will split and hang. The slave also tracks a single outstanding split with no master identifier.
- **No .sdc in the project.** Timing is not signed off by the flow as committed; the numbers in Table 14 come from the constraint applied for this report.
- **ram_4k.v is dead code.** Nothing instantiates it and it is not in the project file list, though its testbench is.

The natural next steps are to connect `decode_err` into the response multiplexer as an error acknowledge, give the arbiter a per-master split owner so Slave 0 can serve both masters, and commit the `.sdc`.

17 Board Bring-Up

The synthesis top is `top_debug`. Twelve pins leave the device; everything else is driven over JTAG.

Two device-wide settings matter on this board. `STRATIX_DEVICE_IO_STANDARD` is set to 3.3-V LVTTTL, since Quartus otherwise defaults the banks to 2.5 V; and `RESERVE_ALL_UNUSED_PINS`

Table 18: Pin assignments (DE2-115, EP4CE115F29C7)

Port	Pin	DE2-115 net	Bank
clk	Y2	CLOCK_50, dedicated clock input	2
rst_n	M23	KEY[0], active low with pull-up	6
ext_rx_serial	AB22	GPIO[0], JP5 — UART RX	4
ext_tx_serial	AC15	GPIO[1], JP5 — UART TX	4
led[0..7]	G19 F19 E19 F21 F18 E18 J19 H19	LEDR[0..7]	7

is set to AS INPUT TRI-STATED, because the DE2-115 hangs SDRAM, SRAM, flash, Ethernet, audio, VGA and HSMC off pins this design does not use, and the default of “as output driving ground” would drive into them.

For the two-board demonstration, connect `ext_tx_serial` (AC15) of each board to `ext_rx_serial` (AB22) of the other, plus a common ground. The pin choices need not match between boards; only the baud rate and frame format do.

```
quartus_map System_Bus_Final --part=EP4CE115F29C7
quartus_fit System_Bus_Final
quartus_asm System_Bus_Final
```

Listing 7: Synthesis, fit and bitstream

A Appendix: Selected RTL

A.1 address_decoder.v

```

1
2 module address_decoder (
3     input wire [13:0] addr,
4     input wire bus_valid,
5     output reg sel_s0, // 0x0000 - 0x0FFF (4K Split)
6     output reg sel_s1, // 0x1000 - 0x1FFF (4K Fast)
7     output reg sel_s2, // 0x2000 - 0x27FF (2K Fast)
8     output reg sel_bridge, // 0x3000 - 0x3FFF (External Bridge)
9     output reg decode_err
10 );
11
12 always @(*) begin
13     sel_s0 = 1'b0;
14     sel_s1 = 1'b0;
15     sel_s2 = 1'b0;
16     sel_bridge = 1'b0;
17     decode_err = 1'b0;
18
19     if (bus_valid) begin
20         if (addr[13:12] == 2'b00) begin
21             sel_s0 = 1'b1; // 0x0000 - 0x0FFF
22         end else if (addr[13:12] == 2'b01) begin
23             sel_s1 = 1'b1; // 0x1000 - 0x1FFF
24         end else if (addr[13:11] == 3'b100) begin
25             sel_s2 = 1'b1; // 0x2000 - 0x27FF
26         end else if (addr[13:12] == 2'b11) begin
27             sel_bridge = 1'b1; // 0x3000 - 0x3FFF
28         end else begin
29             decode_err = 1'b1;
30         end
31     end
32 end
33
34 endmodule

```

A.2 arbiter_2m_split.v

```
1 'timescale 1ns / 1ps
2
3 module arbiter_2m_split (
4     input wire clk,
5     input wire rst_n,
6
7     // Master Requests and Grants
8     input wire req0,
9     input wire req1,
10    output reg gnt0,
11    output reg gnt1,
12
13    // Slave 0 Split Interface
14    input wire s0_split_req,
15    input wire s0_split_done,
16    output reg m0_split_notify,
17    output reg m1_split_notify
18 );
19
20 localparam IDLE = 2'b00;
21 localparam GRANT_M0 = 2'b01;
22 localparam GRANT_M1 = 2'b10;
23
24 reg [1:0] state, next_state;
25 reg m0_split_pending;
26 reg split_data_ready;
27
28 // 1. Sequential State and Status Tracking
29 always @(posedge clk or negedge rst_n) begin
30     if (!rst_n) begin
31         state <= IDLE;
32         m0_split_pending <= 1'b0;
33         split_data_ready <= 1'b0;
34     end else begin
35         state <= next_state;
36
37         // Set when split occurs
38         if (state == GRANT_M0 && s0_split_req) begin
39             m0_split_pending <= 1'b1;
40         end
41
42         // Latch background completion pulse
43         if (s0_split_done) begin
44             split_data_ready <= 1'b1;
45         end
46
47         // Clear split flags when M0 successfully re-acquires the bus and completes
48         if (state == GRANT_M0 && !s0_split_req && split_data_ready) begin
49             m0_split_pending <= 1'b0;
50             split_data_ready <= 1'b0;
51         end
52     end
53 end
54
55 // 2. Combinational Next-State Logic
56 always @(*) begin
57     next_state = state;
58     gnt0 = 1'b0;
59     gnt1 = 1'b0;
60     m0_split_notify = 1'b0;
61     m1_split_notify = 1'b0;
62
63     case (state)
64     IDLE: begin
65         // Priority 1: Service pending split completion
66         if (m0_split_pending && (split_data_ready || s0_split_done)) begin
67             next_state = GRANT_M0;
68         end
69         // Priority 2: Standard Master 0 request
70         else if (req0 && !m0_split_pending) begin
71             next_state = GRANT_M0;
```

```

72         end
73         // Priority 3: Master 1 request
74         else if (req1) begin
75             next_state = GRANT_M1;
76         end
77     end
78
79     GRANT_MO: begin
80         gnt0 = 1'b1;
81         if (s0_split_req) begin
82             m0_split_notify = 1'b1;
83             next_state = req1 ? GRANT_M1 : IDLE;
84         end else if (!req0) begin
85             if (req1)
86                 next_state = GRANT_M1;
87             else
88                 next_state = IDLE;
89         end
90     end
91
92     GRANT_M1: begin
93         gnt1 = 1'b1;
94         if (!req1) begin
95             if (m0_split_pending && (split_data_ready || s0_split_done))
96                 next_state = GRANT_MO;
97             else if (req0 && !m0_split_pending)
98                 next_state = GRANT_MO;
99             else
100                 next_state = IDLE;
101         end
102     end
103
104     default: next_state = IDLE;
105 endcase
106 end
107
108 endmodule

```

A.3 master_node.v

```

1  `timescale 1ns / 1ps
2
3  module master_node (
4      input wire clk,
5      input wire rst_n,
6
7      // Command Interface
8      input wire cmd_start,
9      input wire cmd_we,
10     input wire [13:0] cmd_addr,
11     input wire [7:0] cmd_wdata,
12     output reg [7:0] cmd_rdata,
13     output reg cmd_done,
14
15     // Arbiter Interface
16     output reg bus_req,
17     input wire bus_gnt,
18     input wire bus_split,
19
20     // Shared Bus Interface
21     output reg [13:0] m_addr,
22     output reg [7:0] m_wdata,
23     output reg m_we,
24     output reg m_valid,
25     input wire [7:0] bus_rdata,
26     input wire bus_ready
27 );
28
29     localparam IDLE = 3'b000;
30     localparam REQ = 3'b001;

```

```

31  localparam DRIVE = 3'b010;
32  localparam WAIT = 3'b011;
33  localparam SPLIT_W = 3'b100;
34  localparam DONE = 3'b101;
35
36  reg [2:0] state;
37  reg [13:0] reg_addr;
38  reg [7:0] reg_wdata;
39  reg reg_we;
40
41  always @(posedge clk or negedge rst_n) begin
42      if (!rst_n) begin
43          state <= IDLE;
44          bus_req <= 1'b0;
45          m_valid <= 1'b0;
46          m_addr <= 14'h0000;
47          m_wdata <= 8'h00;
48          m_we <= 1'b0;
49          cmd_rdata <= 8'h00;
50          cmd_done <= 1'b0;
51          reg_addr <= 14'h0000;
52          reg_wdata <= 8'h00;
53          reg_we <= 1'b0;
54      end else begin
55          cmd_done <= 1'b0;
56
57          case (state)
58              IDLE: begin
59                  if (cmd_start) begin
60                      reg_addr <= cmd_addr;
61                      reg_wdata <= cmd_wdata;
62                      reg_we <= cmd_we;
63                      bus_req <= 1'b1;
64                      state <= REQ;
65                  end
66              end
67
68              REQ: begin
69                  if (bus_gnt) begin
70                      m_addr <= reg_addr;
71                      m_wdata <= reg_wdata;
72                      m_we <= reg_we;
73                      m_valid <= 1'b1;
74                      state <= DRIVE;
75                  end
76              end
77
78              DRIVE: begin
79                  if (bus_split) begin
80                      m_valid <= 1'b0;
81                      bus_req <= 1'b1; // Keep requesting for when data becomes ready
82                      state <= SPLIT_W;
83                  end else if (bus_ready) begin
84                      m_valid <= 1'b0;
85                      bus_req <= 1'b0;
86                      // Reads only. Slaves echo din onto dout during a
87                      // write, so capturing unconditionally would clobber
88                      // the last read value with the byte just written.
89                      if (!reg_we) cmd_rdata <= bus_rdata;
90                      cmd_done <= 1'b1;
91                      state <= DONE;
92                  end else begin
93                      state <= WAIT;
94                  end
95              end
96
97              WAIT: begin
98                  if (bus_split) begin
99                      m_valid <= 1'b0;
100                     bus_req <= 1'b1;
101                     state <= SPLIT_W;
102                 end else if (bus_ready) begin
103                     m_valid <= 1'b0;

```

```

104         bus_req <= 1'b0;
105         // Reads only. Slaves echo din onto dout during a
106         // write, so capturing unconditionally would clobber
107         // the last read value with the byte just written.
108         if (!reg_we) cmd_rdata <= bus_rdata;
109         cmd_done <= 1'b1;
110         state <= DONE;
111     end
112 end
113
114     SPLIT_W: begin
115         bus_req <= 1'b1;
116         if (bus_gnt) begin
117             m_addr <= reg_addr;
118             m_wdata <= 8'h00;
119             m_we <= 1'b0;
120             m_valid <= 1'b1;
121             state <= DRIVE;
122         end
123     end
124
125     DONE: begin
126         state <= IDLE;
127     end
128
129     default: state <= IDLE;
130 endcase
131 end
132 end
133
134 endmodule

```

A.4 slave_0_split_4k.v

```

1  `timescale 1ns / 1ps
2
3  module slave_0_split_4k (
4      input wire clk,
5      input wire rst_n,
6      input wire sel,
7      input wire we,
8      input wire [11:0] addr,
9      input wire [7:0] din,
10     output reg [7:0] dout,
11     output reg ready,
12     output reg split_req,
13     output reg split_done
14 );
15
16     reg [2:0] latency_cnt;
17     reg processing_split;
18     reg data_ready_for_fetch;
19     reg [11:0] saved_raddr;
20     reg [7:0] split_data_reg;
21     reg just_completed; // blocks a spurious re-trigger caused by
22                         // master_node holding sel high for one
23                         // extra cycle after it samples 'ready'
24
25     localparam SPLIT_LATENCY = 3'd3;
26
27     // ---- Dedicated M9K block, simple dual-port mode ----
28     // Port A: write-only, driven by the live bus address (this module
29     // never performs a direct read on port A -- every read goes
30     // through the split mechanism on port B).
31     // Port B: read-only, driven by the internally latched saved_raddr,
32     // fetched once the split latency has elapsed.
33     // This maps the two logically-independent accesses the original
34     // behavioral code made into mem[] onto the block's two real ports,
35     // instead of onto extra LUT-based logic.
36     wire [7:0] ram_q_b;

```

```

37
38     altsyncram #(
39         .operation_mode ("DUAL_PORT"),
40         .width_a (8),
41         .widthad_a (12),
42         .numwords_a (4096),
43         .width_b (8),
44         .widthad_b (12),
45         .numwords_b (4096),
46         .lpm_type ("altsyncram"),
47         .outdata_reg_b ("UNREGISTERED"),
48         .address_aclr_b ("NONE"),
49         .outdata_aclr_b ("NONE"),
50         .indata_aclr_a ("NONE"),
51         .wrcontrol_aclr_a ("NONE"),
52         .ram_block_type ("M9K"),
53         .init_file ("UNUSED"),
54         .intended_device_family ("Cyclone IV E")
55     ) s0_ram (
56         .clock0 (clk),
57         .clock1 (clk), // DUAL_PORT mode requires clock1 to be
58                       // explicitly connected even when port B
59                       // shares the same clock as port A
60
61         .clocken0 (1'b1),
62         .clocken1 (1'b1),
63         .aclr0 (1'b0),
64         .aclr1 (1'b0),
65         // Port A -- write path
66         .address_a (addr),
67         .data_a (din),
68         .wren_a (sel & we),
69         .rden_a (1'b1),
70         .q_a (),
71         // Port B -- split-fetch read path
72         .address_b (saved_raddr),
73         .data_b (8'h00),
74         .wren_b (1'b0),
75         .q_b (ram_q_b)
76     );
77
78     always @(posedge clk or negedge rst_n) begin
79         if (!rst_n) begin
80             dout <= 8'h00;
81             ready <= 1'b0;
82             split_req <= 1'b0;
83             split_done <= 1'b0;
84             processing_split <= 1'b0;
85             data_ready_for_fetch <= 1'b0;
86             latency_cnt <= 3'd0;
87             split_data_reg <= 8'h00;
88             saved_raddr <= 12'h000;
89             just_completed <= 1'b0;
90         end else begin
91             ready <= 1'b0;
92             split_req <= 1'b0;
93             split_done <= 1'b0;
94             just_completed <= 1'b0;
95
96             // 1. Bus Access Handling
97             if (sel) begin
98                 if (we) begin
99                     // Fast Single-Cycle Write (handled entirely by port A)
100                     dout <= din;
101                     ready <= 1'b1;
102                 end else begin
103                     // Read Handling
104                     if (data_ready_for_fetch && (addr == saved_raddr)) begin
105                         // Master returned to collect completed split data
106                         dout <= split_data_reg;
107                         ready <= 1'b1;
108                         data_ready_for_fetch <= 1'b0;
109                         just_completed <= 1'b1;
110                     end else if (!processing_split && !data_ready_for_fetch && !just_completed) begin

```

```

110         // Initial Read request: trigger split handshake
111         saved_raddr <= addr;
112         processing_split <= 1'b1;
113         split_req <= 1'b1;
114         latency_cnt <= 3'd1;
115     end
116 end
117 end
118
119 // 2. Background Split Latency Timer
120 if (processing_split) begin
121     if (latency_cnt == SPLIT_LATENCY) begin
122         // saved_raddr has been stable on port B for the whole
123         // latency window, which is well beyond the M9K's
124         // 1-cycle synchronous read latency, so ram_q_b is
125         // guaranteed valid here.
126         split_data_reg <= ram_q_b;
127         split_done <= 1'b1;
128         data_ready_for_fetch <= 1'b1;
129         processing_split <= 1'b0;
130         latency_cnt <= 3'd0;
131     end else begin
132         latency_cnt <= latency_cnt + 1'b1;
133     end
134 end
135 end
136 end
137
138 endmodule

```